

Funciones en C++.

Una función es un bloque de código que realiza alguna operación. Una función puede definir opcionalmente parámetros de entrada que permiten a los llamadores pasar argumentos a la función. Una función también puede devolver un valor como salida. Las funciones son útiles para encapsular las operaciones comunes en un solo bloque reutilizable, idealmente con un nombre que describa claramente lo que hace la función. La siguiente función acepta dos enteros de un llamador y devuelve su suma; a y b son parámetros de tipo **int**.

```
int suma(int a, int b){
    return a + b;
}
```

La función se puede invocar, o llamar, desde cualquier número de lugares en el programa. Los valores que se pasan a la función son los argumentos, cuyos tipos deben ser compatibles con los tipos de parámetro en la definición de función.

```
int main(){
    int i = suma(5, 10);
    int j = suma(i, 35);
    cout << "El valor de j es: " << j << endl; // 50
}
```

Con una función se deben realizar tres operaciones: declaración, definición y llamada o invocación.

Declaración o prototipo de una función.

En C++ es obligatorio usar prototipos. Un prototipo es una declaración de una función. Consiste en una presentación de la función, exactamente con la misma estructura que la definición, pero sin cuerpo y terminada con un ";". La estructura de un prototipo es:

```
[extern|static] <tipo_valor_retorno> [<modificadores>]
<identificador>(<lista_parámetros>);
```

En general, el prototipo de una función se compone de las siguientes secciones:

- ✓ Opcionalmente, una palabra que especifique el tipo de almacenamiento puede ser **extern** o **static**. Si no se especifica ninguna, por defecto será **extern**. No te preocupes de esto todavía, de momento sólo usaremos funciones externas, lo menciono porque es parte de la declaración.
- ✓ El tipo del valor de retorno, que puede ser **void**, si no necesitamos valor de retorno. En C, si no se establece, será **int** por defecto, aunque en general se considera una mala técnica de programación omitir el tipo de valor de retorno de una función. En C++ es obligatorio indicar el tipo del valor de retorno.
- ✓ Modificadores opcionales. Tienen un uso muy específico.

- ✓ El identificador de la función. Es costumbre, muy útil y recomendable, poner nombres que indiquen, lo más claramente posible, qué es lo que hace la función, y que permitan interpretar qué hace el programa con sólo leerlos. Cuando se precisen varias palabras para conseguir este efecto se puede usar alguna de las reglas más usuales. Una consiste en separar cada palabra con un "_". Otra, que yo prefiero, consiste en escribir la primera letra de cada palabra en mayúscula y el resto en minúsculas. Por ejemplo, si hacemos una función que busque el número de teléfono de una persona en una base de datos, podríamos llamarla "busca_telefono" o "BuscaTelefono".
- ✓ Una lista de declaraciones de parámetros entre paréntesis. Los parámetros de una función son los valores de entrada (y en ocasiones también de salida). Para la función se comportan exactamente igual que variables, y de hecho cada parámetro se declara igual que una variable. Una lista de parámetros es un conjunto de declaraciones de parámetros separados con comas. Puede tratarse de una lista vacía. En C es preferible usar la forma "función(void)" para listas de parámetros vacías. En C++ este procedimiento se considera obsoleto, se usa simplemente "función()".

Por ejemplo:

```
int Mayor(int a, int b);
```

Un prototipo sirve para indicar al compilador los tipos de retorno y los de los parámetros de una función, de modo que compruebe si son del tipo correcto cada vez que se use esta función dentro del programa, o para hacer las conversiones de tipo cuando sea necesario.

En el prototipo, los nombres de los parámetros son opcionales, y si se incluyen suele ser como documentación y ayuda en la interpretación y comprensión del programa. El ejemplo de prototipo anterior sería igualmente válido si se escribiera como:

```
int Mayor(int, int);
```

Esto sólo indica que en algún lugar del programa se definirá una función "Mayor" que admite dos parámetros de tipo **int** y que devolverá un valor de tipo **int**. No es necesario escribir nombres para los parámetros, ya que el prototipo no los usa. En otro lugar del programa habrá una definición completa de la función.

Normalmente, los prototipos de las funciones se declaran dentro del fichero del programa, o bien se incluyen desde un fichero externo, llamado fichero de cabecera, (para esto se usa la directiva **#include**).

Ya lo hemos dicho más arriba, pero las funciones son **extern** por defecto. Esto quiere decir que son accesibles desde cualquier punto del programa, aunque se encuentren en otros ficheros fuente del mismo programa.

En contraposición las funciones declaradas **static** sólo son accesibles dentro del fichero fuente donde se definen.

Definición de funciones

Al igual que hemos visto con las variables, las funciones deben declararse, para lo que usaremos los prototipos, pero también deben definirse.

Una definición contiene además las instrucciones con las que la función realizará su trabajo, es decir, su código.

La sintaxis de una definición de función es:

```
[extern|static] <tipo_valor_retorno> [modificadores]
<identificador>(<lista_parámetros>)
{
    [sentencias]
}
```

Como vemos, la sintaxis es idéntica a la del prototipo, salvo que se elimina el punto y coma final, y se añade el cuerpo de función que representa el código que será ejecutado cuando se llame a la función. El cuerpo de la función se encierra entre llaves "{ }".

La definición de la función se hace más adelante o más abajo, según se mire, es decir, se hace después que el prototipo. Lo habitual es hacerlo después de la función **main**. Los parámetros definidos en la cabecera de la función se llaman **parámetros formales**. Son las variables que reciben los valores de los argumentos en la llamada a la función.

El resultado de la función se devuelve a la instrucción que la ha llamado por medio de la sentencia **return**. El tipo_devuelto indica de qué tipo es el dato devuelto por la función. Puede ser cualquier tipo básico, o definido por el usuario, pero **no puede ser un array o una función**. Si no se indica, se supone que es **int**. La sentencia **return** puede aparecer más de una vez en el cuerpo de la función, aunque lo aconsejable es que aparezca sólo una sentencia **return**. Si la función no devuelve ningún valor, se puede omitir o poner simplemente **return**. La sentencia **return** provoca la finalización inmediata de la función en el momento en que se ejecuta.

Llamar o invocar a una función.

Para que una función se ejecute es necesario llamarla o invocarla desde alguna parte del programa.

La llamada a una función está formada por su nombre seguido de una lista de argumentos entre paréntesis y separados por comas (cero o más argumentos) que son los datos que se le envían a la función.

Los argumentos que aparecen en la llamada a una función se llaman parámetros actuales, porque son los valores que se pasan a ésta en el momento de la ejecución.

Los parámetros actuales y los formales deben coincidir en número, orden y tipo. Si el tipo de un parámetro actual no coincide con su correspondiente parámetro formal, el sistema

lo convertirá al tipo de este último, siempre que se trate de tipos compatibles. Si no es posible la conversión, el compilador dará los mensajes de error correspondientes.

Si la función devuelve un valor, la llamada a la función puede estar incluida en una expresión que recoja el valor devuelto.

Cuando se llama a una función, se crean en memoria las variables que aparecen en la cabecera de la función. Las variables declaradas en la cabecera de una función así como las que se declaran en el interior de la misma son variables locales de la función. Esto significa que solo son accesibles dentro de la función. Aunque tengan el mismo nombre que otras variables que puedan aparecer en otras partes del programa se trata de variables distintas. La memoria que ocupan las variables locales de una función se libera cuando acaba la ejecución de la función y dejan de estar accesibles.

Cuando se llama a una función, la ejecución del programa pasa a la función y cuando ésta acaba, la ejecución continúa a partir del punto donde se produjo la llamada.

Ejemplos de funciones en C++

Ejemplo 1

```
funcion1.cpp
1  /*
2  Escriba una función que devuelva el resultado de un
3  número elevado a un exponente dado.
4  Fecha: 08 - junio - 2020
5  Autor: Carlos Benitez
6  */
7  #include<iostream>
8  #include<cmath>
9  int potencia(int b, int e); //prototipo, declaracion
10 using namespace std;
11 int main(){
12     int base;
13     int exponente;
14     int calcular;
15     do{
16         cout<<"Ingrese la base: ";
17         cin>>base;
18     }while(base<=0);
19     do{
20         cout<<"Ingrese el exponente: ";
21         cin>>exponente;
22     }while(exponente<0);
23     calcular = potencia(base,exponente); //invocar o llamar
24     cout<<"El número: "<<base<<" elevado a: "<<exponente<<" = " <<calcular<<endl;
25     return 0;
26 }
27 int potencia(int b, int e){ //definicion
28     int resultado;
29     resultado = pow(b,e);
30     return resultado;
31 }
```

Ejemplo 2

```
funcion2.cpp
1  /*
2  Escriba una función que devuelva el número mayor
3  de un número almacenado en un vector de N elementos.
4  Fecha: 08 - junio - 2020
5  Autor: Carlos Benitez
6  */
7  #include<iostream>
8  #include<cstdlib>
9  #include<ctime>
10 const int N=100;
11 void CargarVector(int vector[N], int limite); //prototipo
12 void MostrarVector(int vector[N], int limite); //prototipo
13 int HallarMayor(int vector[N], int limite); //prototipo, declaracion
14 using namespace std;
15 int main(){
16     int cantidad;
17     int mayor;
18     do{
19         cout<<"Ingrese la cantidad de elementos: ";
20         cin>>cantidad;
21     }while(cantidad<=0);
22     int V[cantidad];
23     CargarVector(V,cantidad); //invocar
24     MostrarVector(V,cantidad); //invocar
25     mayor = HallarMayor(V,cantidad); //invocar o llamar
26     cout<<endl<<"El número mayor es: " << mayor<<endl;
27     return 0;
28 }
29 /*Función para cargar el valor de los elementos del vector*/
30 void CargarVector(int vector[N], int limite){
31     srand(time(NULL));
32     for(int i=0; i < limite; i++){
33         vector[i] = rand()%20+1;
34     }
35 }
36 /*Función para imprimir el valor de los elementos del vector*/
37 void MostrarVector(int vector[N], int limite){
38     for(int i=0; i < limite; i++){
39         cout<<vector[i]<<"\t";
40     }
41 }
42 /*Función para hallar el mayor de los elementos del vector*/
43 int HallarMayor(int vector[N] /* entrada - vector de n enteros */, int limite /* entrada - número de elementos */){
44     int maximo; /* mayor valor en curso */
45     /* Mayor elemento inicial. */
46     maximo = vector[0];
47     /* Compara cada elemento restante con el mayor en curso; guarda el mayor */
48     for(int i=0; i < limite; i++){
49         if (vector[i] > maximo){
50             maximo = vector[i];
51         }
52     }
53     return maximo;
54 }
```

Contenido

Funciones en C++.	1
Declaración o prototipo de una función.	1
Definición de funciones.	3
Llamar o invocar a una función.	3
Ejemplos de funciones en C++	4
Ejemplo 1	4
Ejemplo 2	5